

What Google's security publications can teach the Tetris.js project

Learning only. Nothing here changes the blueprint, the repo or any decision. Prepared October 3, 2026, after a third reading pass that recovered full PDF texts. Every claim comes from a page I fetched; sources are in section 5.

Short answer: yes, and it mostly supports your direction. Google's strongest documented ideas: make a bug class impossible through safe APIs and types so most code needs no security reasoning; keep the code that must be trusted small and review it hard; measure outcomes; and use full proofs on small, well-specified targets such as cryptographic code. Google's own Safe Coding paper describes its approach as *semi-formal*, built on the same ideas as modular proofs (contracts, invariants) but checked by a type checker plus expert review, and says formalizing those contracts is possible in principle.

Limit on what I read: this is not "every Google security paper". No bounded set exists. I read a chosen set of 27 sources: 14 read in full, 9 read in large part, 4 skimmed. My first fetches were cut at about 50,000 characters; I then downloaded full PDFs and chapter pages and read the tails. Exact coverage is in section 1.

1. Coverage, with source type

Google publishes security work across Google Research (one old snapshot of just its security, privacy and abuse area lists 289 publications, with 2018 entries; today's count is higher), the Security Blog, Cloud docs, Chromium and Android docs, Project Zero, OSS-Fuzz, and a book. Papers, technical reports, blogs and product docs carry different weight, so each is labeled. Two of the papers (Ghost, Fiat) are by outside authors about, or shipped by, Google systems.

Depth (count)	Sources (P = paper or article, T = Google technical report, B = blog, D = product or project doc, S = slides, K = book)
Read in full (14)	1 Kern, <i>Safe Coding</i> (ACM Queue, Nov 2025) P. 2 Kern, <i>Securing the Tangled Web</i> (ACM Queue 2014) P. 3 Rebert and Kern, <i>Secure by Design: Google's Perspective on Memory Safety</i> (tech report, Mar 2024) T: full 13-page text from a third-party mirror of the PDF (Google's own page has the abstract only); I read all but a short stretch on hardware features. 4 Kern slides (USENIX Security 2015) S. 5 Chromium Rule of 2 D. 6 Secure by Design blog B. 7 Eliminating Memory Safety Vulnerabilities B. 8 Rust in Android 2025 B. 9 Scaling Memory Safety (giflib, Aug 2026) B. 10 SLSA introduction B. 11 Leveling Up Fuzzing 2024 B. 12 Project Wycheproof B. 13 OSS-Fuzz announcement B. 14 <i>Using Formal Methods at Google</i> (IETF 118) S, which is thin on text.
Read in large part (9)	15 <i>Site Isolation</i> (USENIX Security 2019) P, full 19 pages obtained: read abstract, sections 1, 2, 4.2, 5, 6 and 8; sections 3 and 4.1 only skimmed. 16 <i>CSP Is Dead, Long Live CSP</i> (CCS 2016) P, full 12 pages obtained: read the intro, results, section 4 and conclusion; the study methodology details only skimmed. 17 <i>Ghost in the Android Shell</i> (SOSP 2025) P: Cambridge authors testing Google's pKVM, not a Google paper; 16 pages obtained: read intro, approach, testing, discussion, related work and conclusion; the middle technical sections 3 to 5 only skimmed. 18 <i>Fiat Cryptography</i> (IEEE S&P 2019, MIT authors, shipped in BoringSSL) P, 18 pages: read overview, trusted base, discussion and related work; the Coq derivation details in sections III to V skimmed. 19 BeyondCorp (USENIX ;login: 2014) P, all 6 pages obtained: read the first half and the access-control and migration sections. 20 <i>Building Secure and Reliable Systems</i> K: complete text of ch. 4, 5, 6, 8, 12, 13, 14 obtained (7 of 21 chapters); closely read ch. 6 invariants and trusted base, ch. 8 fail-safe, ch. 12 frameworks and types, ch. 13 testing, static analysis and formal methods, ch. 14 artifact verification; ch. 4 and 5 headings only. 21 OSS-Fuzz USENIX 2017 slides S. 22 Agentic OSS-Fuzz (May 2026) B, about two thirds. 23 Pixel baseband B, first half.
Skimmed (4)	24 BeyondProd D; 25 Binary Authorization for Borg D; 26 Google security overview whitepaper D; 27 Trusted Types guide (web.dev) D.
Not read	Book ch. 1-3, 7, 9-11, 15-21; Project Zero; SAIF; the OSS-Fuzz USENIX full paper (page unavailable, only slides); hundreds of Google Research papers; Chromium security docs beyond Rule of 2; any formal-verification paper not listed above. I did not hit a true access block on any selected source; remaining gaps are reading depth, as labeled.

2. What you can learn, in order of value for your goal

#	Documented practice	What the sources report	What it could mean for Tetris (my hypothesis)
1	Make risky operations unavailable in ordinary code. Wrap them in safe abstractions. (Safe Coding paper, Tangled Web, Kern slides)	A "risky operation" has a safety precondition the caller must meet. Safe code contains none. Each risky operation lives inside a module whose public API has no preconditions for callers. Preconditions are lifted into <i>type invariants</i> of wrapper types (TrustedSqlString, SafeHtml). Composition of safe code and safe abstractions is then checked by the ordinary type checker. Enforcement is by lint or compiler check, with an expert-managed allowlist for exceptions (a simple grep presubmit is called enough for small repos). Reported results: hundreds of web apps on hardened frameworks averaged under one XSS report per year; Google Photos has had none; no reported SQL injection for over a decade across hundreds of apps on the safe query API. One large app went from 31 XSS bugs in 2012 to 4 after strict templates (Kern 2014, called anecdotal by its author).	One constructor and validator is the only way to build a game state or load a save. A lint rule forbids raw sinks (innerHTML, eval, direct JSON.parse of storage) with a tiny reviewed exception list. This matches your "every route into state" obligation.
2	Safe Coding is explicitly semi-formal. (Safe Coding paper, "Formal Foundations")	It reuses modular verification structure (function contracts, invariants) but expresses contracts through type signatures, and relies on rigorous but informal expert reasoning inside each abstraction. The paper says it is "clearly not formally sound" (reflection can break encapsulation) and calls it "soundness". It also says: in principle contracts can be formalized and the abstractions formally verified, which may suit domains where the reasoning is subtle, such as unsafe Rust.	A formally proven version of the small conversion core would be a step the paper describes as possible but does not report doing. This is one paper's scope, not a claim about all Google practice.
3	Name invariants and keep the trusted computing base small. (Book ch. 6, ch. 13)	An invariant holds however the environment misbehaves. The book weighs effort against harm: tests and review give limited confidence; full proof was done for a microkernel at about 20 person-years and is called "typically not feasible" for large application software; its middle path is understandability by design. It says to keep the trusted base as small as possible, with a narrow, clean interface.	Write the core's invariants first (legal position, bounds, score and level rules, save format). They are the base of any proof.
4	Rule of 2: untrustworthy input, unsafe language, high privilege. Pick at most two. (Chromium D)	The doc lists JavaScript as memory-safe. Unsafe code inside a safe language must be small, encapsulated and documented. As memory bugs shrink, attention can shift to application-semantic bugs.	JavaScript covers the memory leg, so remaining risk is semantic: state logic, storage input, DOM sinks.
5	Prevent in new code first; keep mitigations and detection as extra layers. (Secure by Design T, Android blogs)	Android memory-safety share of vulnerabilities: 76% (2019) to 24% (2024), under 20% (2025). Google estimates 1000x lower density in its Rust than in its C/C++ and 4x lower rollback rate. The tech report says bug finding alone does not give acceptable assurance: the libwebp zero-day CVE-2023-4863 was missed despite 97.55% fuzz coverage of the file, and finding bugs does not fix them. The report also says hardware memory tagging (MTE, 16 tags) gives only probabilistic detection (6.25% chance a dangling pointer is missed), and that Rust is the only mature language giving temporal safety without runtime mechanisms; Go and Java can still have data races (syzkaller has 5,000+ kernel bugs found, hundreds open).	Treat any change to the trusted core as a security event that re-opens its proof obligations. Do not read coverage as safety. Android figures are a different domain: a principle, not a Tetris number.
6	Safe by construction is still not absolute. (Rust in Android B, Safe Coding)	A buffer overflow in unsafe Rust (CrabbyAVIF) was caught before shipping, and guard pages made it non-exploitable. About 4% of Android Rust is unsafe. Google keeps defense in depth and found a crash-reporting gap.	Supports keeping containment and adversarial tests next to proofs.
7	Check equivalence by differential testing at scale. (Scaling Memory Safety B)	giflib rewritten in Rust with an LLM: 30 million GIFs compared, 200 million differential-fuzz iterations over six days with no deviations, adversarial LLM review, and expert review of all unsafe code. It found an edge case and an old out-of-bounds write in a legacy patch to the original C code.	Differential runs add evidence next to proofs: they can catch mismatches between a model and an implementation, or a spec error. They do not replace an all-inputs proof.
8	Fuzzing and test suites have defined limits. (OSS-Fuzz S/B, Wycheproof B, Fuzzing 2024 B)	OSS-Fuzz slides: fuzz narrow targets, "the narrower the better", single-process and deterministic; Heartbleed was found by libFuzzer in 10 seconds once it was run. Wycheproof (80+ tests, 40+ bugs found at launch) says passing "does not imply that the library is secure", only that known attacks fail. AI-generated targets found a cJSON bug in a function a human harness already covered.	A pure state-transition function is a good fuzz and property-test target. Keep a list of what each test does and does not rule out.
9	Executable specs, proofs and tests answer different questions. (Ghost paper P, Fiat P, formal-methods S, book ch. 6 and 13)	Cambridge's pKVM work writes a full functional spec in C (about 14,000 lines against about 11,000 lines of pKVM) as a runtime test oracle. It found 5 bugs; only 1 came from running the spec, and the other 4 came from reading the code to write it. Random testing found 9 errors in the spec itself. Cost: about 1 person-year against about 30 for pKVM development. The authors say it "does not" give the assurance of full verification and call writing the spec "a tool for thinking". Fiat Cryptography does go further: it proves the general	For a proof goal these sources suggest complementary evidence: an executable spec checks that the written model matches the code, the proof covers all inputs for the stated model. Fiat's habit of listing unproved assumptions (compiler, printer) matches your explicit-assumptions goal. These sources cover system and crypto components; whole-product proofs are outside this sample, which says

		algorithm once in Coq, derives the code per prime by verified partial evaluation, and ships it in BoringSSL. It states what it does not prove: constant-time behavior of the compiler output is presumed, and the trusted base includes a pretty-printer and the C toolchain. Book ch. 13 says formal methods have high up-front cost because requirements must be written in mathematically rigorous form, and gives hardware and cryptographic domains as successes. Google's formal-methods slides list barriers (prototype tools, proof maintenance, "how do we know the model is correct?").	nothing about all Google work.
9b	Assume a component is already compromised, then contain it. (Site Isolation P)	The paper's design assumes an attacker may control a whole renderer process and still must not read other sites' data. Cost reported: 9-13% total memory overhead; about 20 implementation bugs reported after rollout, none enough to turn it off. It lists residual limits: CORB covers only HTML, XML and JSON responses it can confirm; timing can reveal whether Site Isolation is on.	Hypothesis: state an attacker model explicitly (who controls storage, URL, extensions, same-origin scripts) and say which guarantees hold under it and which do not.
9c	Log-only first, then enforce. (BeyondCorp P, Trusted Types D)	BeyondCorp ran a traffic monitor in logging mode before enforcement mode, and a traffic analysis pipeline before moving users. Trusted Types has a report-only mode. CSP's 'strict-dynamic' paper lists what it still misses: injection into the src of a created script, injection into a nonced script, and scriptless attacks.	Hypothesis: when adding any protective check, record what it would have blocked before turning it on, and write down what it still cannot stop.
10	Check that a protection really protects. (CSP paper P, Trusted Types D)	Of 26,011 distinct real CSP policies from Google's index, 94.72% were trivially bypassable; 90.63% allowed inline scripts or arbitrary hosts; 51.05% of the stricter 9.37% were still bypassable. The authors propose nonce-based policies and 'strict-dynamic'. Trusted Types locks DOM sinks and is rolled out in report-only mode first.	If you add a CSP or Trusted Types, test that a known injection is blocked, and use nonce or hash rules, not host allowlists. Browser support not checked.
11	Verify artifacts, not just people. (SLSA B, BAB D, book ch. 14)	SLSA maps eight attack points with real incidents (SolarWinds, event-stream, CodeCov). Binary Authorization requires review, verifiable builds and provenance before code runs.	For a small static site: few or no dependencies, pinned versions, reproducible build. Google's controls are far larger than needed.
12	Decide fail-open versus fail-closed on purpose. (Book ch. 8)	First set the "minimal nonnegotiable security posture". A system that cannot verify itself should lock down.	Reject a corrupt save without destroying the old one (links to safe write order, O4).
13	Simplicity helps. (Book ch. 12, Rust in Android B)	Avoid deep nesting, remove "just in case" code, repay debt. Rust changes had fewer review rounds and lower rollback rates.	Supports minimizing trusted code before minimizing tests.

3. What probably does not carry over

BeyondCorp, BeyondProd, Binary Authorization and the infrastructure whitepaper answer company-scale access and deployment questions. Site Isolation took about five years and is a browser-process design; its paper's own threat model leaves XSS out of scope. Rust and C++ interop, Scudo, MiraclePtr and OSS-Fuzz C/C++ pipelines do not apply to a JavaScript game. One caution from the agentic OSS-Fuzz post: its helper scripts run coding agents in an unrestricted mode that executes arbitrary code, so use them only in a fully trusted environment.

4. How this compares with the Apple reading

Same direction: small trusted core, scoped assurance with stated assumptions, defense in depth beside proofs, validation at every boundary. **Google adds:** a documented method for making safety a type-level property enforced by lint (and a candid statement that it is semi-formal); differential and executable-spec testing as evidence that complements proofs; measured outcomes; candid cost reports. **Apple adds:** hardware-backed enforcement with no browser-game equivalent. These notes describe the sources I read, not everything either company has published.

5. Sources (fetched October 3, 2026)

Safe Coding (ACM Queue): queue.acm.org/detail.cfm?id=3773098 | Secure by Design tech report (mirror, Google abstract page research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/): elhacker.info/Books/BOOKS%20PART%205/Google_Memory_Safety-.pdf | Secure by Design blog: security.googleblog.com/2024/03/secure-by-design-googles-perspective-on.html | Tangled Web: queue.acm.org/detail.cfm?id=2663760 | Kern slides: usenix.org/sites/default/files/conference/protected-files/sec15_slides_kern.pdf | Rule of 2: chromium.googlesource.com/chromium/src/+HEAD/docs/security/rule-of-2.md | Eliminating Memory Safety Vulnerabilities: security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html | Rust in Android: blog.google/security/rust-in-android-move-fast-fix-things/ | Scaling Memory Safety: bughunters.google.com/blog/scaling-memory-safety | Pixel baseband: blog.google/security/bringing-rust-to-the-pixel-baseband/ | Site Isolation: usenix.org/system/files/sec19-reis.pdf | CSP paper: research.google.com/pubs/archive/45542.pdf | Ghost in the Android Shell: cl.cam.ac.uk/~km569/pubs/sosp25-exec-spec.pdf | Fiat Cryptography: adam.chlipala.net/papers/FiatCryptoSP19/FiatCryptoSP19.pdf | BoringSSL fiat README: boringssl.googlesource.com/boringssl/+HEAD/third_party/fiat/README.chromium | Formal methods slides: datatracker.ietf.org/meeting/118/materials/slides-118-ufmrg-using-formal-methods-at-google | Building Secure and Reliable Systems: google.github.io/building-secure-and-reliable-systems/ (raw/ch04, 05, 06, 08, 12, 13, 14) | OSS-Fuzz slides: usenix.org/sites/default/files/conference/protected-files/usenixsecurity17_slides_serebryany.pdf | OSS-Fuzz announcement: security.googleblog.com/2016/12/announcing-oss-fuzz-continuous-fuzzing.html | Fuzzing 2024: security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html | Agentic OSS-Fuzz: blog.oss-fuzz.com/posts/introducing-agentic-capabilities-in-oss-fuzz/ | Wycheproof: security.googleblog.com/2016/12/project-wycheproof.html | SLSA: security.googleblog.com/2021/06/introducing-slsa-end-to-end-framework.html | Trusted Types: web.dev/articles/trusted-types | BeyondProd: docs.cloud.google.com/docs/security/beyondprod | Binary Authorization for Borg: docs.cloud.google.com/docs/security/binary-authorization-for-borg | Security overview: docs.cloud.google.com/docs/security/overview/whitepaper | BeyondCorp: static.googleusercontent.com/media/research.google.com/en/pubs/archive/43231.pdf | Research area snapshot: research.google.com/pubs/SecurityPrivacyandAbusePrevention.html